[NOMBRE DE LA UNIVERSIDAD]

[NOMBRE DE LA FACULTAD]

Unidad de aprendizaje: Compiladores

Proyecto Integrador Académico (PIA) – AD2026

Chimpan-C

Compilador de un lenguaje simple estilo C

Entregable 1

Semana 1: Definición del lenguaje y planeación

Semana 2: Análisis léxico

Integrantes del equipo

Nombre	**Matrícula**
Alejandro Quintanilla Leal	2010568
Perla Natalíe González Treviño	1903796
Liliana Valeria Ramos Avila	2056778
Angel Emiliano Carmona Lozano	2069119
Luis Rolando Aguilar Ortiz	2121179
Yael Abiezer García Guerra	2050107

Docente: [NOMBRE DEL PROFESOR]

Grupo: [GRUPO] Hora/Salón: [HORA / SALÓN]

[Ciudad], [fecha de entrega]

Contenido

1. Introducción

SEMANA 1 – Definición del lenguaje y planeación

2. Definición del lenguaje Chimpan-C
3. Gramática preliminar
4. Identificación de tokens
5. Ejemplos de entrada válidos e inválidos
6. Alcance del proyecto
7. Roles del equipo
8. Cronograma

SEMANA 2 – Análisis léxico

9. Implementación del analizador léxico
10. Tokens, lexemas y patrones
11. Uso de expresiones regulares
12. Manejo de errores léxicos
13. Pruebas de tokenización
14. Documentación del scanner
15. Conclusiones del entregable
16. Referencias
17. Declaración de integridad académica y originalidad

Anexo A. Código fuente del analizador léxico

1. Introducción

El presente documento corresponde al **Entregable 1** del Proyecto Integrador Académico de la unidad de aprendizaje de Compiladores, el cual agrupa los contenidos de la **Semana 1 (Definición del lenguaje y planeación)** y la **Semana 2 (Análisis léxico)**.

El equipo eligió la opción **"Compilador de un lenguaje simple (expresiones, variables, control de flujo)"**. Para ello se diseñó **Chimpan-C**, un lenguaje imperativo de tipado estático con sintaxis inspirada en C y algunas palabras reservadas con temática de chango. La temática le da identidad al lenguaje sin sacrificar legibilidad: solo cuatro palabras clave son temáticas (`banan`, `jungle`, `throw` y `grab`) y el resto conserva la forma estándar que cualquier programador reconoce.

El compilador se implementa en **Python 3** sin dependencias externas. En este entregable se presenta la especificación del lenguaje, su gramática preliminar, la identificación de tokens, ejemplos de programas válidos e inválidos, el alcance, los roles y el cronograma del equipo, así como la implementación completa y probada del **analizador léxico (scanner)**.

SEMANA 1

Definición del lenguaje y planeación

2. Definición del lenguaje Chimpan-C

2.1 Descripción general

Característica	Decisión de diseño
Nombre	Chimpan-C
Paradigma	Imperativo / procedural
Sintaxis	Estilo C: bloques con llaves `{ }` y sentencias terminadas en `;`
Tipado	Estático y explícito (el tipo se declara junto a la variable)
Punto de entrada	Bloque `jungle { ... }` (equivale a `main`)
Extensión de archivo	`.chc`
Sensibilidad a mayúsculas	Sí: `banan` es palabra reservada, `Banan` es identificador
Comentarios	De línea `// ...` y de bloque `/* ... */`
Lenguaje de implementación	Python 3.10+ (biblioteca estándar)

2.2 Palabras reservadas

Chimpan-C tiene **17 palabras reservadas**. Cuatro de ellas son temáticas y reemplazan a sus equivalentes tradicionales:

Palabra	Categoría	Equivalente	Uso
banan	Temática	var	Declara una variable: `banan int x = 5;`
jungle	Temática	main	Bloque principal del programa
throw	Temática	return	Regresa un valor desde una función
grab	Temática	input / scanf	Lee un valor de la entrada estándar: `grab(x);`
if / else	Control de flujo	—	Selección condicional
while	Control de flujo	—	Ciclo condicional
break / continue	Control de flujo	—	Salir del ciclo / pasar a la siguiente iteración
print	E/S	printf	Imprime uno o más valores: `print(a, " ", b);`
int, float, bool, string	Tipo	—	Tipos de dato primitivos
void	Tipo	—	Tipo de retorno de funciones que no regresan valor
true / false	Literal	—	Literales booleanos

2.3 Tipos de datos

Tipo	Descripción	Ejemplos de literales	Rango / notas
int	Entero con signo de 32 bits	0, 42, 2026	0 a 2 147 483 647 (el signo es el operador unario -)
float	Número real de doble precisión	3.14, 0.5, 70.0	Requiere dígitos a ambos lados del punto
bool	Valor lógico	true, false	Resultado de operadores relacionales y lógicos
string	Cadena de caracteres	"Hello, jungle!"	Comillas dobles; escapes \n \t \r \" \\
void	Sin valor	—	Solo como tipo de retorno de funciones

Reglas de tipos previstas para el análisis semántico (Semana 4): conversión implícita de `int` a `float` en operaciones mixtas; la división entre enteros produce un entero; el operador `+` entre dos `string` concatena; las condiciones de `if` y `while` deben ser de tipo `bool`.

2.4 Operadores y precedencia

Nivel	Operadores	Descripción	Asociatividad
1 (mayor)	( ) f(x)	Agrupación y llamada a función	Izquierda
2	! -	Negación lógica y negativo (unarios)	Derecha
3	* / %	Multiplicación, división, módulo	Izquierda
4	+ -	Suma (o concatenación) y resta	Izquierda
5	< <= > >=	Relacionales	Izquierda
6	== !=	Igualdad y desigualdad	Izquierda
7	&&	Y lógico	Izquierda
8	||	O lógico	Izquierda
9 (menor)	=	Asignación (solo a nivel de sentencia)	Derecha

2.5 Estructura de un programa

Un programa en Chimpan-C está formado por cero o más declaraciones globales (variables y funciones) seguidas obligatoriamente del bloque principal `jungle`. Las funciones se declaran al estilo C, indicando el tipo de retorno, el nombre y sus parámetros tipados:

```
// factorial.chc — función recursiva, if/else y while
int fact(int n) {
    if (n <= 1) {
        throw 1;
    } else {
        throw n * fact(n - 1);
    }
}

jungle {
    banan int i = 1;
    while (i <= 5) {
        print("fact(", i, ") = ", fact(i), "\n");
        i = i + 1;
    }
}
```

Figura 1. Programa factorial.chc: función recursiva, if/else, while y throw.

3. Gramática preliminar

La gramática se expresa en notación **EBNF**: `{ x }` significa cero o más repeticiones, `[ x ]` significa opcional, `|` separa alternativas y los símbolos terminales van entre comillas o en MAYÚSCULAS (categorías de tokens).

```
program        → { global_decl } main_block EOF
global_decl    → var_decl | func_decl
main_block     → "jungle" block

var_decl       → "banan" type IDENT [ "=" expr ] ";"
func_decl      → ret_type IDENT "(" [ params ] ")" block
params         → param { "," param }
param          → type IDENT
ret_type       → type | "void"
type           → "int" | "float" | "bool" | "string"

block          → "{" { statement } "}"
statement      → var_decl
               | ident_stmt
               | if_stmt
               | while_stmt
               | return_stmt
               | print_stmt
               | grab_stmt
               | "break" ";"
               | "continue" ";"
               | block
ident_stmt     → IDENT ( "=" expr | "(" [ args ] ")" ) ";"
if_stmt        → "if" "(" expr ")" block [ "else" ( if_stmt | block ) ]
while_stmt     → "while" "(" expr ")" block
return_stmt    → "throw" [ expr ] ";"
print_stmt     → "print" "(" [ args ] ")" ";"
grab_stmt      → "grab" "(" IDENT ")" ";"
args           → expr { "," expr }

expr           → or_expr
or_expr        → and_expr { "||" and_expr }
and_expr       → eq_expr { "&&" eq_expr }
eq_expr        → rel_expr { ( "==" | "!=" ) rel_expr }
rel_expr       → add_expr { ( "<" | "<=" | ">" | ">=" ) add_expr }
add_expr       → mul_expr { ( "+" | "-" ) mul_expr }
mul_expr       → unary { ( "*" | "/" | "%" ) unary }
unary          → ( "!" | "-" ) unary | primary
primary        → INT_LIT | FLOAT_LIT | STRING_LIT | "true" | "false"
               | IDENT [ "(" [ args ] ")" ]
               | "(" expr ")"
```

Figura 2. Gramática libre de contexto preliminar de Chimpan-C (EBNF).

3.1 Decisiones de diseño de la gramática

- **Precedencia codificada en la gramática:** cada nivel de la tabla de precedencia corresponde a un no terminal (`or_expr` → … → `primary`), por lo que la gramática no es ambigua para expresiones.
- **Sin recursión por la izquierda:** las repeticiones `{ }` reemplazan reglas como `add_expr → add_expr + mul_expr`, lo que permite implementar un **parser**

descendente recursivo LL(1) en la Semana 3.

- **Factorización por la izquierda:** las sentencias que inician con un identificador (asignación y llamada) se agrupan en `ident_stmt`, decidiendo con el siguiente token (`=` o `(`).
- **Sin "else colgante":** los cuerpos de `if`, `else` y `while` son siempre bloques con llaves, por lo que no hay ambigüedad sobre a qué `if` pertenece un `else`. El `else if` se obtiene con `else if_stmt`.
- **Inicio distinguible:** una declaración de variable siempre empieza con `banan` y una de función con un tipo, así que el parser distingue ambas con un solo token de anticipación.
- **Asignación como sentencia:** `=` no forma parte de las expresiones, lo que evita errores como `if (x = 5)`.

Esta gramática es **preliminar**: en la Semana 3 se formalizará (conjuntos FIRST/FOLLOW, verificación LL(1)) y podrá ajustarse según lo que se encuentre en la implementación del parser.

4. Identificación de tokens

A partir de la definición del lenguaje se identificaron las siguientes **categorías léxicas**. En total el scanner reconoce **43 tipos de token** (incluyendo `EOF`).

Categoría	Tipos de token	Lexemas de ejemplo
Palabras reservadas temáticas	BANAN, JUNGLE, THROW, GRAB	banan, jungle, throw, grab
Palabras reservadas estándar	IF, ELSE, WHILE, BREAK, CONTINUE, PRINT	if, else, while, print
Tipos de dato	INT, FLOAT, BOOL, STRING, VOID	int, float, void
Identificadores	IDENT	monkey, total, _tmp, x1
Literales numéricos	INT_LIT, FLOAT_LIT	42, 3.14, 70.0
Literales de cadena	STRING_LIT	"Hello, jungle!\n"
Literales booleanos	TRUE, FALSE	true, false
Operadores aritméticos	PLUS, MINUS, STAR, SLASH, PERCENT	+ - * / %
Operadores relacionales	EQ, NEQ, LT, LTE, GT, GTE	== != < <= > >=
Operadores lógicos	AND, OR, NOT	&& || !
Asignación	ASSIGN	=
Delimitadores	LPAREN, RPAREN, LBRACE, RBRACE, COMMA, SEMICOLON	( ) { } , ;
Fin de archivo	EOF	(marca de fin)

Elementos que el scanner **reconoce pero descarta** (no generan token): espacios, tabuladores, saltos de línea, comentarios de línea `//` y comentarios de bloque `/* */`. Los saltos de línea sí se usan para llevar el conteo de línea y columna de cada token.

5. Ejemplos de entrada válidos e inválidos

5.1 Programas válidos

Los siguientes programas son correctos léxica, sintáctica y semánticamente. Todos se encuentran en `examples/valid/`.

```
// hello.chc — primer programa en Chimpan-C
/* El bloque jungle es el punto de entrada
   del programa (equivale a main). */
jungle {
    banan string monkey = "Chimpancín";
    print("Hello, jungle!\n");
    print("Soy ", monkey, " y tengo hambre de \"bananas\"\n");
}
```

Figura 3. hello.chc: comentarios, cadenas con escapes y print con varios argumentos.

```
// average.chc — lectura con grab, flotantes y operadores lógicos
float average(float a, float b, float c) {
    throw (a + b + c) / 3.0;
}

jungle {
    banan float x;
    banan float y;
    banan float z;
    grab(x);
    grab(y);
    grab(z);

    banan float avg = average(x, y, z);
    banan bool passed = avg >= 70.0 && !(avg > 100.0);

    if (passed) {
        print("Aprobado con ", avg, "\n");
    } else {
        print("Reprobado con ", avg, "\n");
    }
}
```

Figura 4. average.chc: lectura con grab, flotantes, operadores lógicos y if/else.

```
// banana_count.chc – variables globales, %, break, continue y void
banan int total = 0;

void eat(int amount) {
    total = total + amount;
}

jungle {
    banan int day = 0;
    while (true) {
        day = day + 1;
        if (day % 2 == 0) {
            continue;           // los días pares el chango descansa
        }
        eat(3);
        if (total >= 12 || day > 30) {
            break;
        }
    }
    print("Bananas comidas: ", total, " en ", day, " dias\n");
}
```

Figura 5. banana_count.chc: variable global, función void, %, break y continue.

5.2 Programas inválidos

Se prepararon ejemplos inválidos clasificados según la **fase del compilador que debe detectar el error**. Los errores léxicos ya son detectados por el scanner de este entregable; los sintácticos y semánticos son léxicamente correctos y servirán como casos de prueba en las Semanas 3 y 4.

Archivo	Fase	Error esperado
lexical/invalid_chars.chc	Léxica	Caracteres `$`, `#`, `&` y `'` que no pertenecen al alfabeto
lexical/unclosed_string.chc	Léxica	Cadena sin comilla de cierre
lexical/bad_numbers.chc	Léxica	`3.14.15`, `12abc`, `5.` mal formados; entero fuera de rango
lexical/unclosed_comment.chc	Léxica	Comentario `/*` sin `*/`
lexical/misc.chc	Léxica	Identificador de más de 31 caracteres, escape `\c` inválido, letra `ñ`
syntax/missing_semicolon.chc	Sintáctica	Falta `;` y falta `)` en la condición del if
syntax/no_jungle.chc	Sintáctica	El programa no tiene bloque principal `jungle`
semantic/type_mismatch.chc	Semántica	Asignar `string` a `int`; sumar `int` + `bool`
semantic/undeclared.chc	Semántica	Redeclaración de `x` y uso de `y` sin declarar

```
jungle {
    banan int $price = 10;
    banan int stock = 5 # comentario estilo Python
    if (stock > 0 & price > 0) {
        print('sin comillas dobles');
    }
}
```

Figura 6. invalid_chars.chc: errores léxicos por caracteres inválidos.

```
// Léxicamente correcto, pero falta ';' y un paréntesis de cierre
jungle {
    banan int x = 5
    if (x > 3 {
        print(x);
    }
}
```

Figura 7. missing_semicolon.chc: léxicamente válido, error sintáctico.

```
// Léxica y sintácticamente correcto, pero con errores de tipo
jungle {
    banan int bananas = "doce";
    banan bool hungry = 3 + true;
}
```

Figura 8. type_mismatch.chc: léxica y sintácticamente válido, error semántico.

6. Alcance del proyecto

6.1 Incluido en el proyecto

- Analizador léxico con expresiones regulares, reporte de errores con línea y columna, y recuperación.
- Analizador sintáctico descendente recursivo (LL(1)) que construye un **AST**.
- Tabla de símbolos con **alcances anidados** (global, función y bloque) y verificación de tipos.
- Detección de errores léxicos, sintácticos y semánticos con mensajes claros.
- Generación de **código de tres direcciones (TAC)** para expresiones, asignaciones, if/else, while, break/continue y llamadas a función.
- Optimizaciones básicas sobre el TAC: **propagación/plegado de constantes** y **eliminación de código muerto**.
- Intérprete del TAC para **ejecutar** los programas y mostrar sus resultados (incluye `print` y `grab`).
- Pruebas automatizadas por fase y documentación técnica.

6.2 Fuera del alcance

- Arreglos, estructuras, punteros, clases y manejo de memoria dinámica.
- Ciclo `for`, `switch` y operadores compuestos (`+=`, `++`).
- Conversión explícita de tipos (casting) y sobrecarga de funciones.
- Generación de código máquina o ensamblador real (opcional en el PIA; se evaluará si el tiempo lo permite).
- Forma SSA (opcional en el PIA; se considerará como mejora).

6.3 Fases del compilador

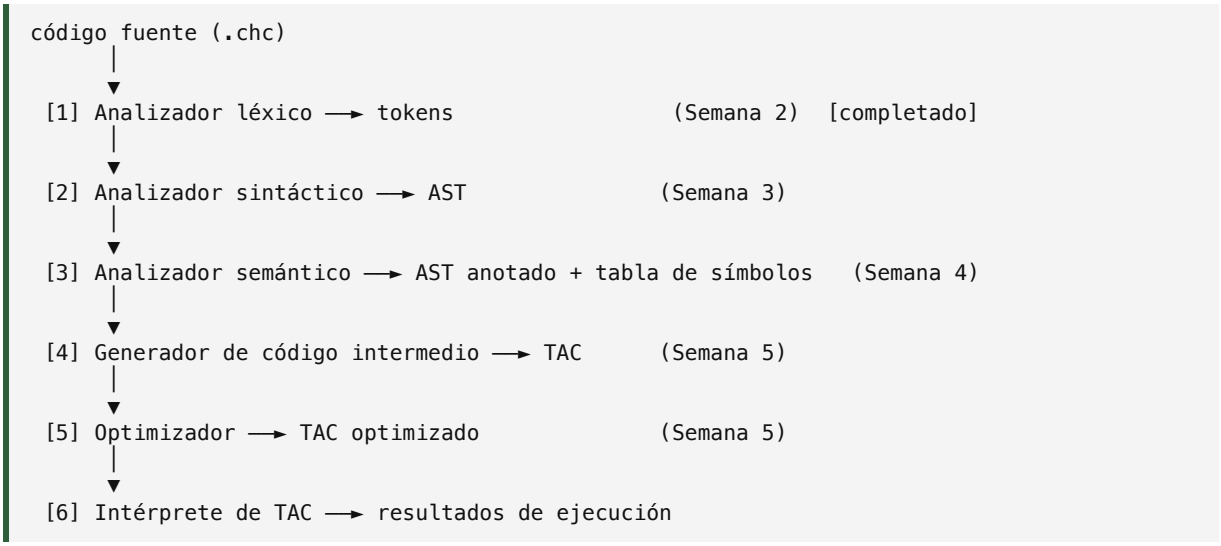


Figura 9. Arquitectura por fases del compilador Chimpan-C.

7. Roles del equipo

Cada integrante tiene un **rol principal**, del cual es responsable de entregar y explicar, y un **rol de apoyo**. Todos los integrantes participan en las pruebas y en la revisión del código de los demás.

Integrante	Matrícula	Rol principal	Rol de apoyo
Alejandro Quintanilla Leal	2010568	Líder de proyecto e integración: coordina tareas, administra el repositorio y une las fases del compilador	Video explicativo y exposición final
Perla Natalíe González Treviño	1903796	Diseño del lenguaje y gramática formal (EBNF, FIRST/FOLLOW)	Documentación técnica del proyecto
Liliana Valeria Ramos Avila	2056778	Analizador léxico (scanner) y manejo de errores léxicos	Pruebas automatizadas
Angel Emiliano Carmona Lozano	2069119	Analizador sintáctico (parser) y construcción del AST	Manejo de errores sintácticos
Luis Rolando Aguilar Ortiz	2121179	Análisis semántico: tabla de símbolos, alcances y tipos	Casos de prueba semánticos
Yael Abiezer García Guerra	2050107	Generación de código intermedio (TAC) y optimización	Intérprete / ejecución del TAC

8. Cronograma

Las fechas se ajustarán al calendario oficial del curso.

Semana	Fechas	Actividades	Responsable(s)	Estado
1	[dd/mm] – [dd/mm]	Definición del lenguaje, gramática preliminar, tokens, ejemplos, alcance, roles y cronograma	Perla (líder de la fase), Alejandro, todo el equipo	Completado
2	[dd/mm] – [dd/mm]	Analizador léxico, expresiones regulares, errores léxicos, pruebas y documentación del scanner	Liliana (líder de la fase), Perla	Completado
—	[dd/mm]	Entrega del Entregable 1 (Semanas 1 y 2)	Alejandro	En curso
3	[dd/mm] – [dd/mm]	Gramática formal, parser LL(1), árbol de derivación, AST y errores sintácticos	Angel, Perla	Pendiente
4	[dd/mm] – [dd/mm]	Tabla de símbolos, alcances, verificación de tipos, errores semánticos y AST anotado	Luis, Angel	Pendiente
5	[dd/mm] – [dd/mm]	Generación de TAC, estructuras de control, optimizaciones e intérprete	Yael, Luis	Pendiente
—	[dd/mm]	Entrega del Entregable 2 (Semanas 3, 4 y 5)	Alejandro	Pendiente
Final	[dd/mm] – [dd/mm]	Integración, pruebas completas, documentación final, video (5 min) y exposición	Todo el equipo	Pendiente

Mecánica de trabajo: el código se versiona en un repositorio Git compartido; cada fase se desarrolla en una rama propia y se integra a la rama principal solo cuando sus pruebas pasan. El equipo tiene una reunión breve de seguimiento cada semana.

SEMANA 2

Análisis léxico

9. Implementación del analizador léxico

El analizador léxico (scanner) es la primera fase del compilador. Recibe el código fuente como una cadena de caracteres y produce una **secuencia de tokens**, cada uno con su tipo, lexema, valor, línea y columna. También detecta y reporta los errores léxicos.

9.1 Organización del código

Archivo	Contenido
chimpanc/tokens.py	Enumeración `TokenType`, tablas `KEYWORDS`, `OPERATORS`, `DELIMITERS` y la clase `Token`
chimpanc/errors.py	Catálogo de errores léxicos (LEX001 – LEX007) y la clase `LexicalError`
chimpanc/lexer.py	Especificación de patrones, expresión regular maestra y la clase `Lexer`
chimpanc/__main__.py	Interfaz de línea de comandos: `python -m chimpanc archivo.chc`
tests/test_lexer.py	31 pruebas automatizadas de tokenización (unittest)

9.2 Algoritmo

1. Se combinan todos los patrones en una **expresión regular maestra** con grupos con nombre.
2. Desde la posición actual se aplica `re.match`; el nombre del grupo que coincidió (`match.lastgroup`) indica la categoría del lexema.
3. Según la categoría: se descarta (espacios y comentarios), se crea un token o se registra un error.
4. Los identificadores se buscan en la tabla `KEYWORDS`; si están, se emite la palabra reservada correspondiente.
5. Los literales se convierten a su valor (`int`, `float`, `bool`, o `str` con los escapes resueltos).
6. Se actualizan línea y columna (los comentarios de bloque pueden abarcar varias líneas) y se avanza al final de la coincidencia.
7. Al terminar el texto se agrega el token `EOF`. El resultado es la tupla `(tokens, errores)`.

Como el último patrón (`MISMATCH`) coincide con cualquier carácter, **siempre** hay una coincidencia en cada posición: el scanner nunca se queda atascado y recorre el archivo completo en una sola pasada, en tiempo lineal respecto al tamaño de la entrada.

10. Tokens, lexemas y patrones

Un **token** es la categoría léxica (p. ej. `IDENT`), un **lexema** es la secuencia concreta de caracteres en el código fuente (p. ej. `monkey`) y un **patrón** es la regla, expresada como expresión regular, que describe todos los lexemas válidos de un token.

Token(s)	Patrón (expresión regular)	Lexemas de ejemplo
IDENT / palabras reservadas	[A-Za-z_][A-Za-z0-9_]*	monkey, x1, banan, while
INT_LIT	\d+(?![\w.])	0, 42, 2026
FLOAT_LIT	\d+\.\d+(?![\w.])	3.14, 70.0
STRING_LIT	"(?:[^"\\\n]|\\.)*"	"hola", "a\tb\n"
EQ NEQ LTE GTE AND OR	==|!=|<=|>=|&&|||	== != <= >= && ||
PLUS MINUS STAR SLASH PERCENT LT GT NOT ASSIGN	[+\-*/%<>!=]	+ - * / % < > ! =
LPAREN RPAREN LBRACE RBRACE COMMA SEMICOLON	[(){},;]	( ) { } , ;
(comentario de línea)	//[^\n]*	// nota
(comentario de bloque)	/*[\s\S]*?*/	/* ... */
(espacios)	[\t\r\f]+ y \n	espacio, tabulador, salto de línea

Patrones adicionales que existen **únicamente para detectar errores** (ver sección 12): `BAD_NUMBER \d[\w.]*`, `UNCLOSED_STRING "[^\n]*`, `UNCLOSED_COMMENT /\*[\s\S]*` y `MISMATCH .`.

10.1 Estructura de un token

```
@dataclass(frozen=True)
class Token:
    type: TokenType   # categoría léxica, p. ej. TokenType.IDENT
    lexeme: str       # texto exacto en el código fuente
    value: object     # valor interpretado (int, float, str, bool) o None
    line: int         # línea donde inicia (base 1)
    column: int       # columna donde inicia (base 1)
```

11. Uso de expresiones regulares

El scanner usa el módulo estándar `re` de Python. La especificación completa de patrones es una lista ordenada de pares (nombre, expresión regular):

```
TOKEN_SPEC = [
    ("BLOCK_COMMENT",    r"/\*[\s\S]*?\*/"),
    ("UNCLOSED_COMMENT", r"/\*[\s\S]*"),
    ("LINE_COMMENT",     r"//[^\n]*"),
    ("NEWLINE",          r"\n"),
    ("SKIP",             r"[ \t\r\f]+"),
    ("FLOAT_LIT",        r"\d+\.\d+(?![\w.])"),
    ("INT_LIT",          r"\d+(?![\w.])"),
    ("BAD_NUMBER",       r"\d[\w.]*"),
    ("STRING_LIT",       r'"(?:[^"\\\n]|\\.)*"'),
    ("UNCLOSED_STRING",  r'"[^\n]*'),
    ("IDENT",            r"[A-Za-z_][A-Za-z0-9_]*"),
    ("OPERATOR",         r"==|!=|<=|>=|&&|\|\||[+\-*/%<>!=]"),
    ("DELIMITER",        r"[(){},;]"),
    ("MISMATCH",         r"."),
]

MASTER_PATTERN = re.compile("|".join(f"(?P<{name}>{pattern})" for name, pattern in
TOKEN_SPEC))
```

Figura 10. Especificación de tokens y expresión regular maestra (chimpanc/lexer.py).

11.1 Decisiones importantes

- **El orden resuelve conflictos.** El operador `|` de las expresiones regulares de Python prueba las alternativas de izquierda a derecha. Por eso los comentarios van antes que el operador `/`, `FLOAT_LIT` antes que `INT_LIT`, y los operadores de dos caracteres (`<=`) antes que los de uno (`<`).
- **Máxima coincidencia (maximal munch).** El patrón de identificador consume la secuencia alfanumérica más larga posible, de modo que `banana` es un `IDENT` y no la palabra reservada `banan` seguida de `a`.
- **Palabras reservadas por tabla.** En lugar de un patrón por palabra reservada, se reconoce un identificador y se consulta el diccionario `KEYWORDS` (búsqueda O(1)). Esto simplifica la expresión regular y facilita agregar palabras nuevas.
- **Lookahead negativo.** El patrón `(?![\w.])` garantiza que un número no esté pegado a letras o puntos. Sin él, `12abc` se tokenizaría como `12` seguido de `abc`; con él, el texto completo cae en `BAD_NUMBER` y se reporta un solo error claro.
- **Cuantificador perezoso en comentarios.** `/\*[\s\S]*?\*/` termina en el **primer** `*/`; con `*` voraz se “comería” el código entre dos comentarios distintos.
- **Cadenas con escapes.** `(?:[^"\\\n]|\\.)*` acepta cualquier carácter excepto comilla, diagonal invertida o salto de línea, o bien una diagonal invertida seguida de cualquier carácter; así `\"` no cierra la cadena.

- **Relación con autómatas.** Cada patrón es un lenguaje regular y equivale a un autómata finito; la expresión maestra corresponde a la unión de dichos autómatas, que el motor de `re` recorre desde cada posición del texto.

12. Manejo de errores léxicos

Cada error léxico tiene un **código único**, un mensaje descriptivo y la **línea y columna** donde ocurre. Para los errores más comunes se agrega una sugerencia.

Código	Error	Ejemplo	Mensaje / sugerencia
LEX001	Carácter inválido	$ # @ & ' ñ	Indica el carácter; sugiere &&, ||, comillas dobles o comentarios // según el caso
LEX002	Cadena sin cerrar	"Chimpancín;	Falta la comilla de cierre antes del fin de línea
LEX003	Comentario sin cerrar	/* nunca cierra	Falta */ para cerrar el comentario
LEX004	Número mal formado	3.14.15 12abc 5.	El lexema no es un número válido
LEX005	Identificador demasiado largo	(32+ caracteres)	Indica la longitud y el máximo (31)
LEX006	Entero fuera de rango	99999999999	Excede el máximo permitido (2 147 483 647)
LEX007	Escape inválido	"C:\carpeta"	Lista los escapes permitidos: \n \t \r \" \\

12.1 Estrategia de recuperación

El scanner usa una recuperación en **modo pánico a nivel de lexema**: al encontrar un error lo registra, descarta el lexema problemático y **continúa** analizando desde el siguiente carácter. De esta forma se reportan **todos** los errores léxicos de un archivo en una sola ejecución, en lugar de detenerse en el primero.

- Un carácter inválido se descarta solo (por ejemplo, en `$price` se reporta `$` y luego se reconoce `price` como identificador).
- Una cadena sin cerrar descarta el resto de la línea; el análisis continúa en la línea siguiente.
- Un comentario sin cerrar consume hasta el fin del archivo (no es posible saber dónde debía terminar).
- En un escape inválido la cadena se conserva como token y solo se reporta la secuencia incorrecta.

Salida real del scanner con el archivo `invalid_chars.chc` (Figura 6):

```
5 error(es) léxico(s):
  Error léxico [LEX001] línea 2, columna 15: Carácter inválido. '$' no pertenece al
alfabeto de Chimpan-C.
  Error léxico [LEX001] línea 3, columna 25: Carácter inválido. '#' no pertenece al
alfabeto de Chimpan-C. Los comentarios en Chimpan-C usan // o /* */.
  Error léxico [LEX001] línea 4, columna 19: Carácter inválido. '&' no pertenece al
alfabeto de Chimpan-C. ¿Quisiste escribir '&&'?
  Error léxico [LEX001] línea 5, columna 15: Carácter inválido. ''' no pertenece al
alfabeto de Chimpan-C. Las cadenas en Chimpan-C usan comillas dobles (").
  Error léxico [LEX001] línea 5, columna 35: Carácter inválido. ''' no pertenece al
alfabeto de Chimpan-C. Las cadenas en Chimpan-C usan comillas dobles (").
```

Figura 11. Reporte de errores léxicos con código, posición y sugerencia.

13. Pruebas de tokenización

Las pruebas se automatizaron con el módulo estándar `unittest` (`tests/test_lexer.py`). Se ejecutan con `python -m unittest -v`. Se agrupan en cinco categorías:

#	Grupo	Entrada de prueba	Resultado esperado	Res.
1	Reservadas	banan jungle throw grab	BANAN JUNGLE THROW GRAB EOF	Pasó
2	Reservadas	banana	IDENT (no BANAN): máxima coincidencia	Pasó
3	Reservadas	Banan IF	IDENT IDENT (sensible a mayúsculas)	Pasó
4	Literales	42 / 3.14	INT_LIT=42 / FLOAT_LIT=3.14	Pasó
5	Literales	"hola\n\t\"chango\"\\"	STRING_LIT con escapes resueltos	Pasó
6	Literales	true false	TRUE=True, FALSE=False	Pasó
7	Operadores	== != <= >= && ||	EQ NEQ LTE GTE AND OR	Pasó
8	Operadores	x<=10&&!y	IDENT LTE INT_LIT AND NOT IDENT (sin espacios)	Pasó
9	Posiciones	a /* uno\ndos\ntres */ b	b en línea 3, columna 9	Pasó
10	Errores	banan int $x = 1;	LEX001	Pasó
11	Errores	print("hola);	LEX002	Pasó
12	Errores	x; /* sin cierre	LEX003	Pasó
13	Errores	3.14.15 12abc 5.	LEX004 ×3	Pasó
14	Errores	32 letras / 31 letras	LEX005 / sin error (límite exacto)	Pasó
15	Errores	2147483648 / 2147483647	LEX006 / sin error (límite exacto)	Pasó
16	Errores	"hola\q"	LEX007	Pasó
17	Recuperación	banan int @x = 5;	1 error y 7 tokens correctos	Pasó
18	Archivos	examples/valid/*.chc	0 errores en los 4 programas válidos	Pasó
19	Archivos	examples/invalid/lexical/*.chc	≥ 1 error en cada archivo	Pasó
20	Archivos	invalid/syntax, invalid/semantic	0 errores léxicos (se detectan después)	Pasó

Resultado de la ejecución de la batería completa:

```
----------------------------------------------------------------------
Ran 31 tests in 0.002s

OK
```

13.1 Ejemplo de tokenización completa

Salida del scanner para el programa `hello.chc` (Figura 3). Se observa que los comentarios y espacios se descartan, que las palabras reservadas se distinguen de los identificadores y que las cadenas incluyen su valor con los escapes ya resueltos:

```
#  LÍNEA:COL  TIPO         LEXEMA                VALOR
------------------------------------------------------------------
   1  4:1        JUNGLE       jungle
   2  4:8        LBRACE       {
   3  5:5        BANAN        banan
   4  5:11       STRING       string
   5  5:18       IDENT        monkey
   6  5:25       ASSIGN       =
   7  5:27       STRING_LIT   "Chimpancín"           'Chimpancín'
   8  5:39       SEMICOLON    ;
   9  6:5        PRINT        print
  10  6:10       LPAREN       (
  11  6:11       STRING_LIT   "Hello, jungle!\n"     'Hello, jungle!\n'
  12  6:29       RPAREN       )
  13  6:30       SEMICOLON    ;
  14  7:5        PRINT        print
  15  7:10       LPAREN       (
  16  7:11       STRING_LIT   "Soy "                 'Soy '
  17  7:17       COMMA        ,
  18  7:19       IDENT        monkey
  19  7:25       COMMA        ,
  20  7:27       STRING_LIT   " y tengo hambre de \"bananas\"\n" ' y tengo hambre de
"bananas"\n'
  21  7:61       RPAREN       )
  22  7:62       SEMICOLON    ;
  23  8:1        RBRACE       }
  24  9:1        EOF

Total de tokens: 24
Análisis léxico completado sin errores.
```

Figura 12. Tabla de tokens generada para hello.chc.

14. Documentación del scanner

14.1 Requisitos e instalación

- Python 3.10 o superior. No requiere instalar librerías externas.
- Desde la carpeta raíz del proyecto (`chimpan-c/`) se ejecutan todos los comandos.

14.2 Uso desde línea de comandos

```
# Tabla de tokens
python -m chimpanc examples/valid/factorial.chc

# Tokens en formato JSON (útil para depurar o para otras herramientas)
python -m chimpanc examples/valid/factorial.chc --json

# Programa con errores léxicos (termina con código de salida 1)
python -m chimpanc examples/invalid/lexical/bad_numbers.chc

# Ejecutar las pruebas
python -m unittest -v
```

14.3 Uso como biblioteca (para el parser de la Semana 3)

```
from chimpanc import tokenize, TokenType

tokens, errors = tokenize('banan int x = 5;')
for tok in tokens:
    print(tok)          # <BANAN, 'banan', 1:1>  <INT, 'int', 1:7> ...

if errors:
    for err in errors:
        print(err)      # Error léxico [LEX001] línea 1, columna 3: ...
```

14.4 Interfaz pública

Elemento	Descripción
tokenize(source)	Analiza una cadena y regresa (lista de Token, lista de LexicalError). La lista de tokens siempre termina en EOF.
Lexer(source).tokenize()	Equivalente orientado a objetos; permite consultar lexer.tokens y lexer.errors.
Token	type, lexeme, value, line, column.
LexicalError	code, message, lexeme, line, column. str(err) produce el mensaje completo.
TokenType	Enumeración de los 43 tipos de token.
MAX_IDENT_LENGTH / MAX_INT	Límites configurables: 31 caracteres y 2^31 – 1.

14.5 Cómo extender el scanner

- **Nueva palabra reservada:** agregar el tipo en `TokenType` y el lexema en `KEYWORDS`; no hay que tocar las expresiones regulares.
- **Nuevo operador:** agregarlo al patrón `OPERATOR` (los de dos caracteres al inicio) y a la tabla `OPERATORS`.
- **Nuevo error:** registrar el código en `LEXICAL_ERRORS` y agregar su caso en `Lexer._handle`, junto con su prueba.

15. Conclusiones del entregable

En la Semana 1 se definió completamente el lenguaje **Chimpan-C**: sus palabras reservadas, tipos, operadores con su precedencia, la estructura de un programa y una gramática preliminar en EBNF diseñada para ser analizada con un parser LL(1). También se delimitó el alcance, se asignaron roles y se estableció el cronograma del proyecto.

En la Semana 2 se implementó un analizador léxico funcional basado en expresiones regulares, que reconoce 43 tipos de token, registra la posición exacta de cada uno, detecta siete tipos de errores léxicos con mensajes útiles y se recupera de ellos para reportarlos todos en una sola pasada. Las 31 pruebas automatizadas pasan correctamente.

Siguiente paso (Semana 3): formalizar la gramática (FIRST/FOLLOW), implementar el parser descendente recursivo que consumirá la lista de tokens de este scanner y construir el AST.

16. Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2.ª ed.). Pearson Educación.

Nystrom, R. (2021). Crafting Interpreters. Genever Benning. https://craftinginterpreters.com

Python Software Foundation. (2026). re — Regular expression operations. https://docs.python.org/3/library/re.html

Python Software Foundation. (2026). unittest — Unit testing framework. https://docs.python.org/3/library/unittest.html

[Agregar aquí cualquier otra fuente o herramienta de apoyo utilizada, conforme al reglamento de la institución.]

17. Declaración de integridad académica y originalidad

Los abajo firmantes, integrantes del equipo que desarrolla el proyecto **Chimpan-C** para la unidad de aprendizaje de Compiladores (AD2026), declaramos que:

1. El diseño del lenguaje, la gramática, el código fuente, los ejemplos y los análisis presentados en este documento son producto del trabajo del equipo.
2. Toda fuente externa consultada (libros, documentación, sitios web o herramientas) se encuentra debidamente referenciada.
3. No hemos copiado código ni documentación de otros equipos o proyectos sin el reconocimiento correspondiente.
4. Conocemos que el plagio o el uso indebido de código será sancionado conforme al reglamento institucional.
5. Cada integrante conoce y puede explicar el funcionamiento de las partes del proyecto entregadas.

Nombre	**Matrícula**	**Firma**
Alejandro Quintanilla Leal	2010568	
Perla Natalíe González Treviño	1903796	
Liliana Valeria Ramos Avila	2056778	
Angel Emiliano Carmona Lozano	2069119	
Luis Rolando Aguilar Ortiz	2121179	
Yael Abiezer García Guerra	2050107	

[Ciudad], a [fecha].

Anexo A. Código fuente del analizador léxico

El código completo, los ejemplos y las pruebas se entregan junto con este documento en la carpeta `chimpan-c/`.

A.1 chimpanc/lexer.py

```
"""
Analizador léxico (scanner) de Chimpan-C.

Estrategia:
  1. Cada categoría léxica se describe con una expresión regular.
  2. Todas se combinan en una sola expresión maestra con grupos con nombre
     (?P<NOMBRE>patrón). El orden importa: en cada posición gana la primera
     alternativa que coincide, por eso los patrones más específicos van primero
     (comentarios antes que '/', flotantes antes que enteros, etc.).
  3. Se recorre el código fuente de izquierda a derecha con re.match en la
     posición actual (principio de "máxima coincidencia" por patrón).
  4. Las palabras reservadas se reconocen como identificadores y luego se
     buscan en la tabla KEYWORDS.
  5. Los errores no detienen el análisis: se registran y el scanner continúa
     desde el siguiente carácter (recuperación en "modo pánico").
"""

import re

from .errors import LexicalError
from .tokens import DELIMITERS, KEYWORDS, OPERATORS, Token, TokenType

MAX_IDENT_LENGTH = 31
MAX_INT = 2**31 - 1
VALID_ESCAPES = {"n": "\n", "t": "\t", "r": "\r", '"': '"', "\\": "\\"}

# Especificación de tokens: (nombre del grupo, expresión regular)
TOKEN_SPEC = [
    ("BLOCK_COMMENT",    r"/\*[\s\S]*?\*/"),
    ("UNCLOSED_COMMENT", r"/\*[\s\S]*"),
    ("LINE_COMMENT",     r"//[^\n]*"),
    ("NEWLINE",          r"\n"),
    ("SKIP",             r"[ \t\r\f]+"),
    ("FLOAT_LIT",        r"\d+\.\d+(?![\w.])"),
    ("INT_LIT",          r"\d+(?![\w.])"),
    ("BAD_NUMBER",       r"\d[\w.]*"),
    ("STRING_LIT",       r'"(?:[^"\\\n]|\\.)*"'),
    ("UNCLOSED_STRING",  r'"[^\n]*'),
    ("IDENT",            r"[A-Za-z_][A-Za-z0-9_]*"),
    ("OPERATOR",         r"==|!=|<=|>=|&&|\|\||[+\-*/%<>!=]"),
    ("DELIMITER",        r"[(){},;]"),
    ("MISMATCH",         r"."),
]

MASTER_PATTERN = re.compile("|".join(f"(?P<{name}>{pattern})" for name, pattern in
TOKEN_SPEC))

# Sugerencias para caracteres inválidos comunes
HINTS = {
    "&": "¿Quisiste escribir '&&'?",
    "|": "¿Quisiste escribir '||'?",
    "'": "Las cadenas en Chimpan-C usan comillas dobles (\").",
    "#": "Los comentarios en Chimpan-C usan // o /* */.",
}
```

```
class Lexer:
    def __init__(self, source: str):
        self.source = source
        self.tokens: list[Token] = []
        self.errors: list[LexicalError] = []
        self.line = 1
        self.line_start = 0  # índice del primer carácter de la línea actual

    def tokenize(self) -> tuple[list[Token], list[LexicalError]]:
        pos = 0
        while pos < len(self.source):
            match = MASTER_PATTERN.match(self.source, pos)
            kind = match.lastgroup
            lexeme = match.group()
            column = pos - self.line_start + 1
            self._handle(kind, lexeme, column)
            self._advance_lines(lexeme, pos)
            pos = match.end()

        eof_column = len(self.source) - self.line_start + 1
        self.tokens.append(Token(TokenType.EOF, "", None, self.line, eof_column))
        return self.tokens, self.errors

    # ------------------------------------------------------------------
    def _handle(self, kind: str, lexeme: str, column: int) -> None:
        if kind in ("SKIP", "NEWLINE", "LINE_COMMENT", "BLOCK_COMMENT"):
            return
        if kind == "IDENT":
            self._identifier(lexeme, column)
        elif kind == "INT_LIT":
            self._integer(lexeme, column)
        elif kind == "FLOAT_LIT":
            self._add(TokenType.FLOAT_LIT, lexeme, float(lexeme), column)
        elif kind == "STRING_LIT":
            self._string(lexeme, column)
        elif kind == "OPERATOR":
            self._add(OPERATORS[lexeme], lexeme, None, column)
        elif kind == "DELIMITER":
            self._add(DELIMITERS[lexeme], lexeme, None, column)
        elif kind == "BAD_NUMBER":
            self._error("LEX004", f"'{lexeme}' no es un número válido.", lexeme, column)
        elif kind == "UNCLOSED_STRING":
            self._error("LEX002", "Falta la comilla de cierre (\") antes del fin de
línea.",
                        lexeme, column)
        elif kind == "UNCLOSED_COMMENT":
            self._error("LEX003", "Falta '*/' para cerrar el comentario.", lexeme[:2],
column)
        elif kind == "MISMATCH":
            hint = HINTS.get(lexeme, "")
            if not lexeme.isascii():
                hint = "Fuera de cadenas solo se permiten caracteres ASCII (sin acentos ni
ñ)."
            self._error("LEX001", f"'{lexeme}' no pertenece al alfabeto de Chimpan-C.
{hint}".strip(),
                        lexeme, column)

    def _identifier(self, lexeme: str, column: int) -> None:
        if lexeme in KEYWORDS:
            token_type = KEYWORDS[lexeme]
            value = {"true": True, "false": False}.get(lexeme)
            self._add(token_type, lexeme, value, column)
        elif len(lexeme) > MAX_IDENT_LENGTH:
```

```
            self._error("LEX005", f"'{lexeme[:12]}...' tiene {len(lexeme)} caracteres "
                        f"(máximo {MAX_IDENT_LENGTH}).", lexeme, column)
        else:
            self._add(TokenType.IDENT, lexeme, None, column)

    def _integer(self, lexeme: str, column: int) -> None:
        value = int(lexeme)
        if value > MAX_INT:
            self._error("LEX006", f"{lexeme} excede el máximo permitido ({MAX_INT}).",
lexeme, column)
        else:
            self._add(TokenType.INT_LIT, lexeme, value, column)

    def _string(self, lexeme: str, column: int) -> None:
        # Se interpreta el contenido entre comillas resolviendo secuencias de escape
        content = lexeme[1:-1]
        value = []
        i = 0
        while i < len(content):
            char = content[i]
            if char == "\\":
                escape = content[i + 1]
                if escape in VALID_ESCAPES:
                    value.append(VALID_ESCAPES[escape])
                else:
                    self._error("LEX007", f"'\\{escape}' no es un escape válido "
                                "(se permiten \\n \\t \\r \\\" \\\\).",
                                f"\\{escape}", column + 1 + i)
                    value.append(escape)
                i += 2
            else:
                value.append(char)
                i += 1
        self._add(TokenType.STRING_LIT, lexeme, "".join(value), column)

    # ------------------------------------------------------------------
    def _add(self, token_type: TokenType, lexeme: str, value, column: int) -> None:
        self.tokens.append(Token(token_type, lexeme, value, self.line, column))

    def _error(self, code: str, message: str, lexeme: str, column: int) -> None:
        self.errors.append(LexicalError(code, message, lexeme, self.line, column))

    def _advance_lines(self, lexeme: str, start: int) -> None:
        # Actualiza el contador de líneas (los comentarios de bloque pueden abarcar
varias)
        newlines = lexeme.count("\n")
        if newlines:
            self.line += newlines
            self.line_start = start + lexeme.rindex("\n") + 1


def tokenize(source: str) -> tuple[list[Token], list[LexicalError]]:
    """Atajo: analiza `source` y regresa (tokens, errores)."""
    return Lexer(source).tokenize()
```

A.2 chimpanc/tokens.py

```
"""
Definición de tokens del lenguaje Chimpan-C.

Contiene:
  - TokenType: enumeración de todas las categorías léxicas.
  - KEYWORDS: tabla de palabras reservadas (lexema -> tipo de token).
```

```
  - OPERATORS / DELIMITERS: tablas de símbolos (lexema -> tipo de token).
  - Token: estructura que representa un token reconocido por el scanner.
"""

from dataclasses import dataclass
from enum import Enum, auto


class TokenType(Enum):
    # --- Palabras reservadas temáticas de Chimpan-C ---
    BANAN = auto()      # declaración de variable   (equivale a "var")
    JUNGLE = auto()     # bloque principal           (equivale a "main")
    THROW = auto()      # retorno de función         (equivale a "return")
    GRAB = auto()       # lectura de entrada         (equivale a "input")

    # --- Palabras reservadas estándar ---
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    BREAK = auto()
    CONTINUE = auto()
    PRINT = auto()

    # --- Tipos de dato ---
    INT = auto()
    FLOAT = auto()
    BOOL = auto()
    STRING = auto()
    VOID = auto()

    # --- Literales ---
    INT_LIT = auto()
    FLOAT_LIT = auto()
    STRING_LIT = auto()
    TRUE = auto()
    FALSE = auto()

    # --- Identificadores ---
    IDENT = auto()

    # --- Operadores aritméticos ---
    PLUS = auto()       # +
    MINUS = auto()      # -
    STAR = auto()       # *
    SLASH = auto()      # /
    PERCENT = auto()    # %

    # --- Operadores relacionales ---
    EQ = auto()         # ==
    NEQ = auto()        # !=
    LT = auto()         # <
    LTE = auto()        # <=
    GT = auto()         # >
    GTE = auto()        # >=

    # --- Operadores lógicos ---
    AND = auto()        # &&
    OR = auto()         # ||
    NOT = auto()        # !

    # --- Asignación ---
    ASSIGN = auto()     # =

    # --- Delimitadores ---
    LPAREN = auto()     # (
```

```
    RPAREN = auto()     # )
    LBRACE = auto()     # {
    RBRACE = auto()     # }
    COMMA = auto()      # ,
    SEMICOLON = auto()  # ;

    # --- Fin de archivo ---
    EOF = auto()


KEYWORDS = {
    "banan": TokenType.BANAN,
    "jungle": TokenType.JUNGLE,
    "throw": TokenType.THROW,
    "grab": TokenType.GRAB,
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "break": TokenType.BREAK,
    "continue": TokenType.CONTINUE,
    "print": TokenType.PRINT,
    "int": TokenType.INT,
    "float": TokenType.FLOAT,
    "bool": TokenType.BOOL,
    "string": TokenType.STRING,
    "void": TokenType.VOID,
    "true": TokenType.TRUE,
    "false": TokenType.FALSE,
}

OPERATORS = {
    "==": TokenType.EQ,
    "!=": TokenType.NEQ,
    "<=": TokenType.LTE,
    ">=": TokenType.GTE,
    "&&": TokenType.AND,
    "||": TokenType.OR,
    "+": TokenType.PLUS,
    "-": TokenType.MINUS,
    "*": TokenType.STAR,
    "/": TokenType.SLASH,
    "%": TokenType.PERCENT,
    "<": TokenType.LT,
    ">": TokenType.GT,
    "!": TokenType.NOT,
    "=": TokenType.ASSIGN,
}

DELIMITERS = {
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
    ";": TokenType.SEMICOLON,
}


@dataclass(frozen=True)
class Token:
    """Unidad léxica reconocida por el analizador léxico."""

    type: TokenType
    lexeme: str      # texto exacto tal como aparece en el código fuente
    value: object    # valor interpretado (int, float, str, bool) o None
```

```
    line: int        # línea donde inicia el token (base 1)
    column: int      # columna donde inicia el token (base 1)

    def __str__(self) -> str:
        return f"<{self.type.name}, '{self.lexeme}', {self.line}:{self.column}>"
```

A.3 chimpanc/errors.py

```
"""
Errores léxicos del compilador Chimpan-C.

Cada error tiene un código único (LEXnnn) para poder documentarlo y probarlo.
"""

from dataclasses import dataclass

# Catálogo de errores léxicos: código -> descripción general
LEXICAL_ERRORS = {
    "LEX001": "Carácter inválido",
    "LEX002": "Cadena sin cerrar",
    "LEX003": "Comentario de bloque sin cerrar",
    "LEX004": "Número mal formado",
    "LEX005": "Identificador demasiado largo",
    "LEX006": "Entero fuera de rango",
    "LEX007": "Secuencia de escape inválida",
}


@dataclass(frozen=True)
class LexicalError:
    """Error detectado durante el análisis léxico."""

    code: str
    message: str
    lexeme: str
    line: int
    column: int

    def __str__(self) -> str:
        title = LEXICAL_ERRORS[self.code]
        return (f"Error léxico [{self.code}] línea {self.line}, columna {self.column}: "
                f"{title}. {self.message}")
```